

Theory:

1) Global scope refers to variables and functions declared outside any function or block which can be accessed from many parts of the program. Global scope pollution is bad because it can cause:

- * Variable name conflicts
- * Accidental modification of variables
- * Difficult debugging
- * Poor code maintainability

Therefore, unnecessary global variables should be avoided.

2) Function scope means that a variable declared inside a function can only be accessed within that function. It cannot be directly accessed outside the function.

3) Block scope means that a variable declared inside a block `{ }` can only be accessed within that block.

- * `let` → Block-scoped
- * `const` → Block-scoped
- * `var` → Not block scoped; it is function scoped.

4) Lexical scoping means that variable accessibility is determined by where function and variables are written in the source code. The scope chain works by searching for a variable from:

- 1) Current scope
- 2) Outer scope
- 3) Further outer scope

4) Global Scope. An inner function can access variables from its outer lexical scopes.

5) A closure is a function that remembers and can access variables from its outer lexical scope even after the outer function has finished executing.

6) One practical use case is data privacy. A closure can keep a variable private and allow access or modification only through specific functions. Other examples include counters, memoization, debounce functions and event handlers.

7) Closures remember variables because the inner function maintains access to the outer function's lexical environment. As long as the inner function is still reachable and needs those variables, JavaScript keeps that environment available.

8) With `var`, all callbacks share the same loop variable. When `setTimeout` callbacks run, the loop has already finished, so they usually access the final value. Not fine. This because it creates a separate block-scoped binding for each loop iteration. Therefore, each callback remembers its own value.

9) Yes. A closure can contribute to a memory leak when it unnecessarily keeps reference to large objects, DOM elements, or data that are no longer needed. This usually happens when long-lived closures, event listeners, timers or global reference prevent unused objects from being

garbage collected. A closure itself is not automatically a memory task.

10) No, scope and closure are not the same.

* scope determines where a variable can be accessed.

* closure allows a function to remember and access variables from its outer lexical scope. A closure works because of lexical scoping, but scope and closure are different concepts.

Output prediction:

11) 10

12) 1
2

13) 3
3
3

14) 0
1
2

15) 2
1

16) Hi sam

17) deep

18) 2

19) 2

20) 2
1

Error Finding:

21) x is declared inside the outer() function. Therefore, it is only accessible within that function's scope. Trying

to access x outside the function causes a Reference Error because x does not exist in the outer scope.

22) Var is function-scoped, so all callbacks share the same loop variable, when the callbacks execute, the loop has already completed, and they all access the final value. Using let creates a separate block-scoped binding for each iteration. Therefore, every callback remembers its own iteration value.

23) Count ++ is a postfix increment. It first returns the current value and then increments count. Therefore the returned values are:

0
1
2
Even though ~~it~~ internally count becomes:

1
2
3

If ++ count is used instead, it increments first and then returns the new value.

24) Every closure can retain its own lexical environment. If a large number of closures are repeatedly created and stored, they consume additional memory. This can become a problem in performance-critical or frequently executed code because unnecessary closures and their captured variables may remain in memory while still referenced.

25) A variable declared with `let` exists only inside the block `{ }` where it is declared. When accessed outside the block, JavaScript cannot find the variable, resulting in a Reference Error.

26) If a closure references a large object, JavaScript may keep the object in memory as long as the closure is still reachable. Even when the object is no longer needed, it cannot be garbage collected if the closure still holds a reference to it. This can cause unnecessary memory usage and potentially a memory leak.

27) Normally each call to an outer function creates a new lexical environment with its own independent variables. Therefore two closures created by separate function calls should have separate private states. If they share state, it usually means the variable is declared in a shared outer scope, such as a global variable or another common parent scope.

28) Variable shadowing occurs when an inner scope declares a variable with the same name as a variable in an outer scope. The inner variable hides the outer variable within that scope. If the inner variable is accessed before it is properly initialized or assigned, it may cause unexpected behavior or an error instead of using the outer variable.

29) Var is function scoped and does not respect block scope. Therefore a variable declared with var can become accessible throughout the entire function instead of being limited to a specific block. Using let provides block scope and better control over variable visibility, helping maintain intended encapsulation.

30) If the outer function does not return the inner function, the result of calling the outer function will be undefined. Therefore chaining cannot continue because there is no returned function to call again. The inner function must be explicitly returned so that it remains accessible and can be called or chained.

Coding:

```
31) function counter () {  
    let count = 0;  
    return [  
        increment: function () {  
            count ++;  
            return count;  
        },  
        decrement: function () {  
            count --;  
            return count;  
        }  
    ]  
}
```

```

const c = counter();
console.log(c.increment()); // 1
console.log(c.increment()); // 2
console.log(c.decrement()); // 1

```

32) Broken version - var

```

for (var i = 0; i < 3; i++) {
  set Timeout(function() {
    console.log(i);
  }, 1000);
}

```

O/P
3
3
3

Fixed version - let

```

for (let i = 0; i < 3; i++) {
  set Timeout(function() {
    console.log(i);
  }, 1000);
}

```

O/P
0
1
2

33) function makeMultiplier(x) {
 return function(number) {
 return x * number;
 };
}

```

const double = makeMultiplier(2);
console.log(double(5)); // 10
const triple = makeMultiplier(3);
console.log(triple(5)); // 15

```

34) function createBankAccount (initialBalance) {

let balance = initialBalance;

return {

deposit: function (amount) {

if (amount > 0) {

balance += amount;

}

return balance;

};

withdraw: function (amount) {

if (amount > 0 && amount <= balance) {

balance -= amount;

}

else {

console.log("Insufficient balance");

}

return balance;

},

getBalance: function () {

return balance;

}

};

}

const account = createBankAccount(1000);

console.log(account.deposit(500)); // 1500

console.log(account.withdraw(300)); // 1200

console.log(account.getBalance()); // 1200

```

35) if (true) {
    let message = "Hello";
    console.log(message);
}
// console.log(message);
// Reference Error

```

O/P Hello

message cannot be accessed outside the if block.

```

36) function callCounter () {
    let count = 0;
    return function () {
        count++;
        return count;
    };
}
const counterCalls = callCounter();
console.log(countCalls()); // 1
console.log(countCalls()); // 2
console.log(countCalls()); // 3

```

```

37) function memoizeDouble () {
    const cache = {};
    return function (number) {
        if (cache[number] !== undefined) {
            return cache[number];
        }
        const result = number * 2;
        cache[number] = result;
        return result;
    };
}

```

```
const double = memoizeDouble();  
console.log(double(5)); // 10  
console.log(double(5)); // 10 (from cache)  
console.log(double(10)); // 20
```

```
38) function outer() {  
  let message = "Hello";  
  function middle() {  
    function inner() {  
      console.log(message);  
    }  
    inner();  
  }  
  middle();  
}  
outer(); // Hello O/p: Hello
```

```
39) function once(fn) {  
  let hasRun = false;  
  return function () {  
    if (!hasRun) {  
      hasRun = true;  
      return fn();  
    }  
  };  
}  
function greet() {  
  console.log("Hello");  
}  
const greetOnce = once(greet);
```

```
greetOnce(); // Hello  
greetOnce(); // Nothing  
greetOnce(); // Nothing
```

```
10) function createCounter() {  
    let count = 0;  
    return function () {  
        count++;  
        return count;  
    };  
}
```

```
const counter1 = createCounter();
```

```
const counter2 = createCounter();
```

```
console.log(counter1()); // 1
```

```
console.log(counter1()); // 2
```

```
console.log(counter2()); // 1
```

```
console.log(counter2()); // 2
```

This proves that counter1 and counter2 have separate private states.

Real World Scenario:

41) A closure allows each button instance to have its own private ~~own~~ count variable. Each button can update and remember its own count without affecting other buttons. This provides independent state without using a shared global variable.

42) A closure-based module or factory pattern fits this requirement. The balance is kept private inside the outer function, while only selected function such as deposit() and withdraw() are returned. This provides encapsulation and controlled access to the private balance.

43) Older Javascript commonly used var, which is function-scoped. All loop callbacks could therefore share the same loop variable. By the time asynchronous callbacks or event handlers executed, the loop variable often had its final value. Before let, developers commonly used IIFEs or additional closures to create separate values for each iteration.

44) A closure-based factory pattern fits. The outer function stores the request count in a private variable and the returned function remembers and updates that count. This avoids using global variables and keeps each rate limiter's state independent.

45) Look for:

- * Closures referencing removed DOM elements
- * Event listeners that are never removed
- * Timers or intervals that are never cleared
- * Large objects unnecessarily captured by closures
- * Long-lived closures that are no longer needed
- * Reference stored globally or in persisted arrays

The main goal is to identify unnecessary reference that prevent garbage collection.

46) A debounce function needs to remember the same timer ID between ~~new~~ multiple function calls. The closure preserves the timer variable, allowing the previous timer to be cancelled and a new timer to be created. Without remembering the timer between calls, the debounce function cannot correctly delay execution.

47) An IIFE creates a private scope, and closures allow selected returned functions to access variables inside that scope. Variables inside the IIFE cannot normally be accessed directly from outside. Therefore developers could expose only specific functions while keeping internal variables private.

48) A closure can preserve a private cache object inside an outer function. The returned function checks the cache for a previous result before performing the calculation again. Because the closure remembers the cache between calls previously calculated results can be reused. This improves performance for repeated calculations.

49) Each component may represent a separate piece of functionality and require its own independent state. Using a global counter would cause both components to share and modify the same value. Independent closure provides

- * Separate state

- * Better encapsulation

- * No interference between components

- * Less global scope pollution

- * Easier maintenance

50) Closures are tested frequently because they demonstrate understanding of several important Javascript concepts together, including:

- * Scope

- * Lexical scoping

- * Scope chain

- * Functions

- * State preservation

- * Callbacks

- * Asynchronous programming

- * Data privacy

- * Memory management

Closures are also widely used in real-world Javascript features such as event handlers, counters, memoization, debounce functions, modules, and callbacks. Therefore, understanding closures shows a strong understanding of how Javascript functions and variable environments work.